THE UNIVERSITY OF QUEENSLAND
AUSTRALIA
CREATE CHANGE

# Tired of compatibility headache?

# let's try containers.

Date: 04 June 2026


***Jijo Derick Abraham***

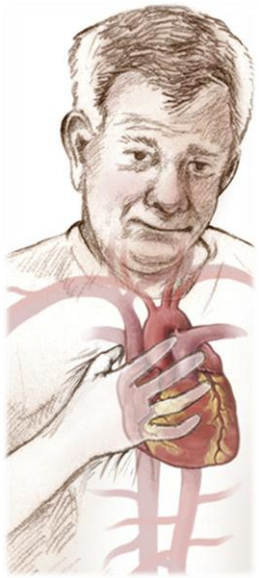
***(Postdoctoral research fellow)***
***Computational Multiphysics Laboratory (CML),***
***School of Mechanical and Mining Engineering, The University of Queensland.***

# About me and my research

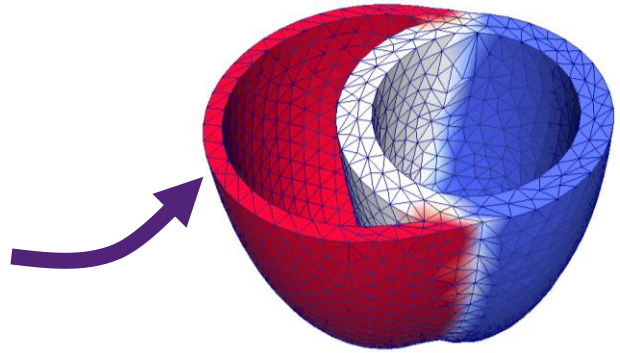
THE UNIVERSITY OF QUEENSLAND AUSTRALIA

## Jijo Derick Abraham

| | | |
|---|---|---|
| 2026-present | : | **Postdoctoral research fellow**<br>University of Queensland, Australia, |
| 2020-2026 | : | **Joint PhD, Computational cardiac mechanics**<br>University of Queensland, Australia, &<br>Indian Institute of Technology Delhi, India |
| 2019-2020 | : | **Junior Research Fellow**<br>(FSI solvers using OpenFOAM)<br>Visvesvaraya National Institute of Technology, Nagpur, India |
| 2017-2019 | : | **M.Tech, Heat Power Engineering**<br>Visvesvaraya National Institute of Technology, Nagpur, India |
| 2012-2016 | : | **B.Tech, Mechanical Engineering**<br>Cochin University of Science and Technology, India |

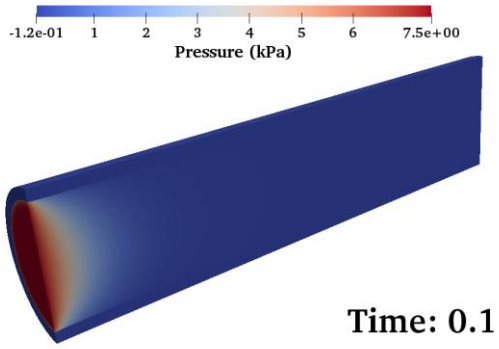


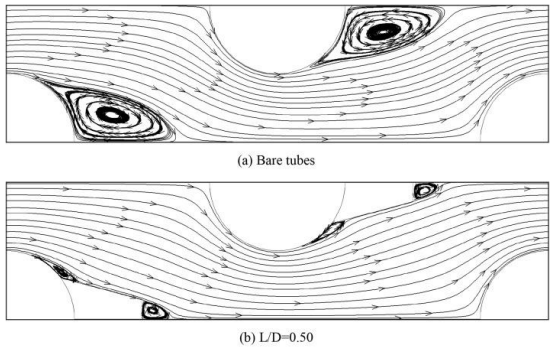
*Heart modelling*



*Rosalia et al. 2021, JACC Basic Transl Sci*

FENICS PROJECT




*Fluid-structure interaction*

SimVascular

*Heat and fluid flow*

OpenFOAM

# Content

- Why we need containers?
- What is a container?
- How does the container work?
- Types of containers
- Working with docker
- Working with singularity
- Drawbacks of containers

For slides and demonstration files
https://github.com/jijoderick/CFD_Tea_discussion_UQ_SOMME_Jijo .git

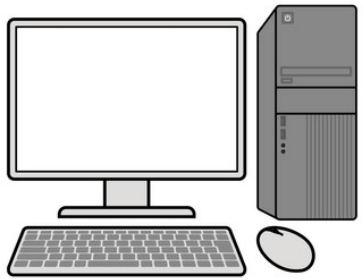
# Why we (CFD researchers) need containers?

**Situations in which we face compatibly headache**

- While trying a new/old code in your field
- While transferring your code to a new team member with a different machine
- When you want to run your code in a new machine or cluster

**Works on one machine, but not on others**

Laptop
(Different OS)



Office PC
(Different Setup)


HPC/Cloud
(Different Env.)

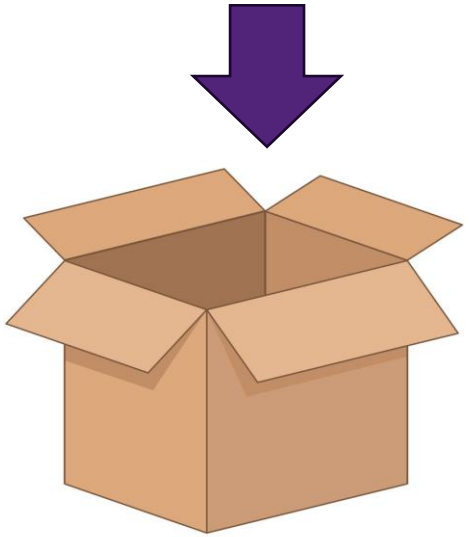
Dependency conflicts
Inconsistent environments
Hard to reproduce results
Time wasted on setup & debugging

*Low productivity, Not reproducible, Hard to scale*

*image sources: VectorStock, shuttlestock*

# How container solves the compatibility headache?

Your CFD application/solver code
System libraries
Dependencies
System tools
Runtime
Others



Contain package your CFD code with everything it need to run consistently, anywhere

**With containers: build once, run anywhere**

Cluster

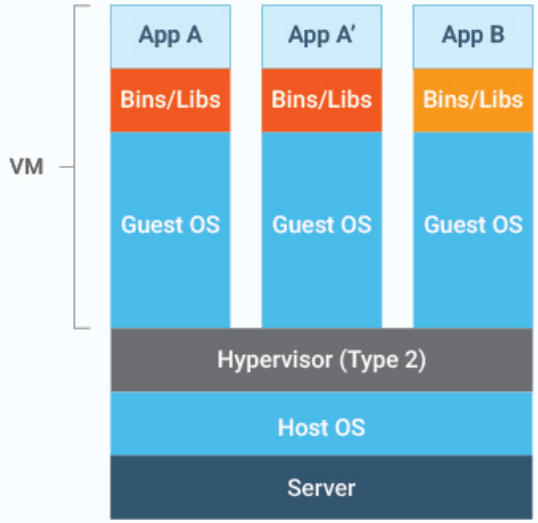
- Consistent environment
- No dependency conflicts
- Portable
- Save time and boost productivity

*image sources: VectorStock, shuttlestock*

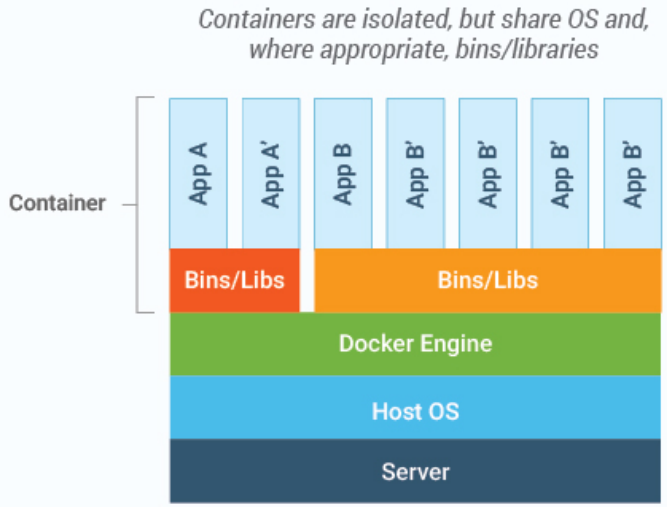
# How does containers work?

THE UNIVERSITY OF QUEENSLAND AUSTRALIA

## Virtual machines




## Containers

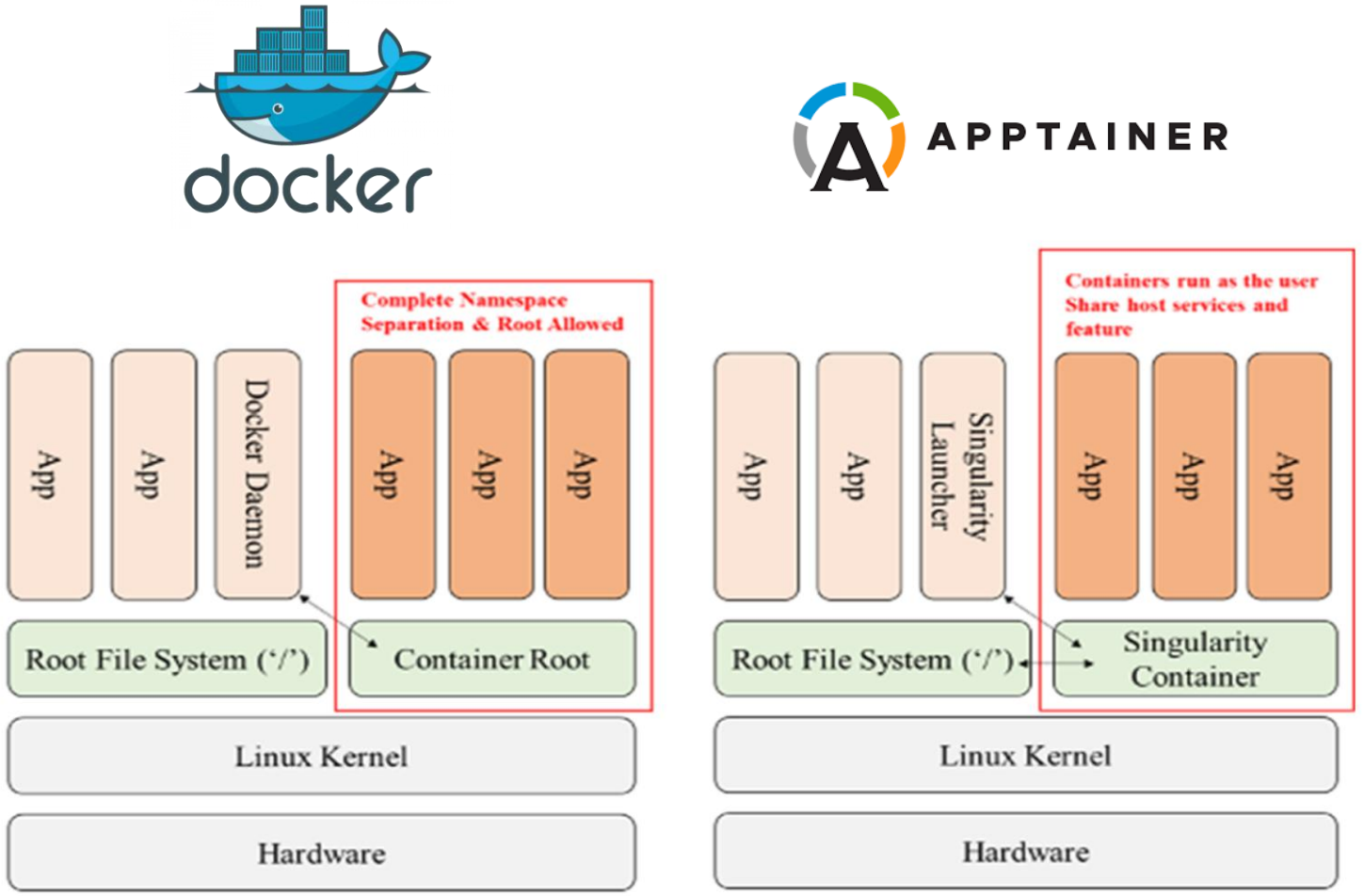



https://www.eginnovations.com/blog/containers-vs-vms/

| Virtual machine | Containers |
|---|---|
| Runs on its own OS | Share host OS |
| Slow boot (minutes) | Fast boot (milliseconds) |
| Limited performance | Native performance |
| Large image size | Small image size |
| Require more memory | Require less memory |
| Hard to use more VMs in one machine | Easy to use many containers in one machine |
| More secure.<br>Fully isolated | Less secure.<br>Process level security |

https://ipcisco.com/lesson/container-vs-virtual-machine/

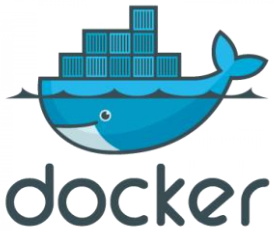
# Types of containers




*Le et al. Appl. Sci.* **2021**, *11*(3), 923; https://doi.org/10.3390/app11030923

# Comparison of containers





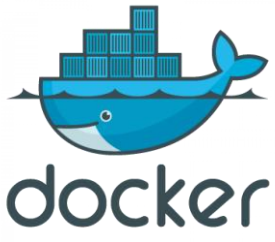


| Feature | Docker | Singularity (APPTAINER) |
|---|---|---|
| Security | Less secure, runs as root in default | More secure |
| User access | Requires root privileges | Users can run directly.<br>No root privilege required. |
| Best for | Development, testing, local environments | HPC, research computing, shared clusters |
| Performance | Excellent | Better on HPC |
| Image format | Layered images | Single file images |

**Take away:** Use docker for local development, and use singularity for secure, reproducible and HPC environments

# Part -A

# Docker workflow



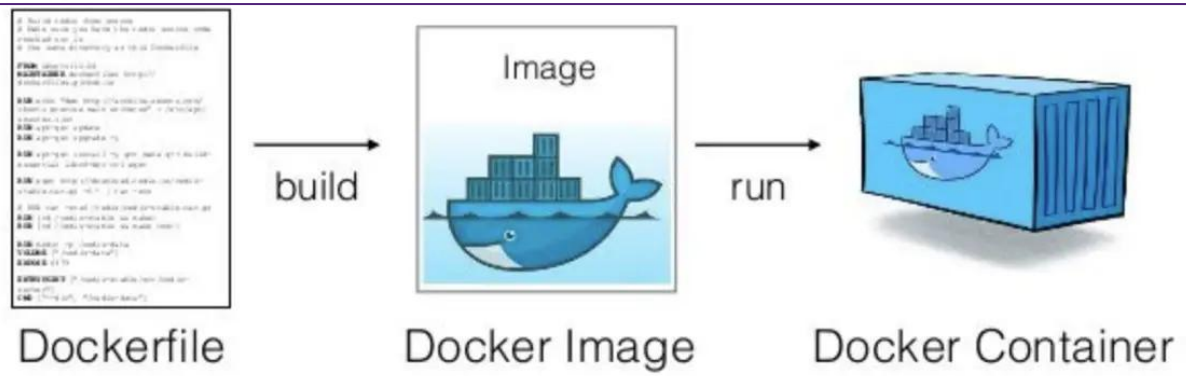
# Steps of making a docker container

To install docker engine (https://docs.docker.com/engine/install/ )

sudo apt install docker.io

Check the status

sudo systemctl status docker




*https://www.mlwithramin.com/blog/docker*

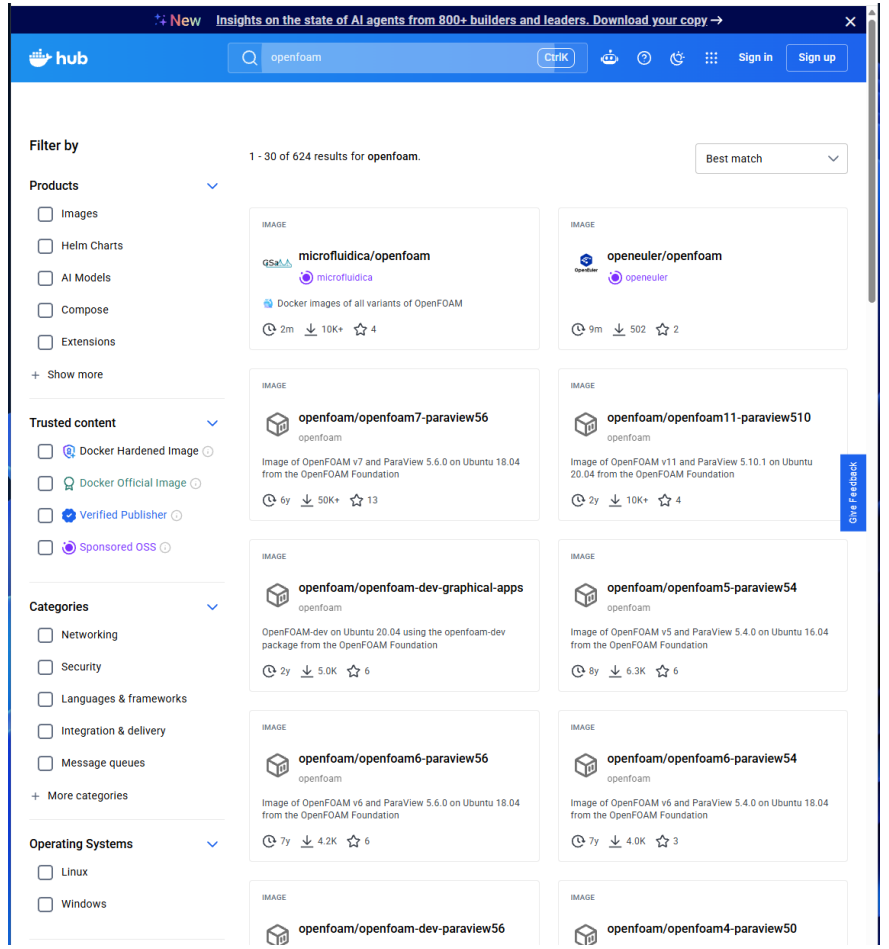
To download the image from docker image repositories such as,

Docker Hub, Quay, Biocontainers, Rocker, Jupyter containers, GitHub Container Registry, and Gitlab Container Registry

To pull an image

sudo docker pull image repository

Eg: sudo docker pull fenics/stable:py3




*https://hub.docker.com/*

# Building a docker image from Dockerfile

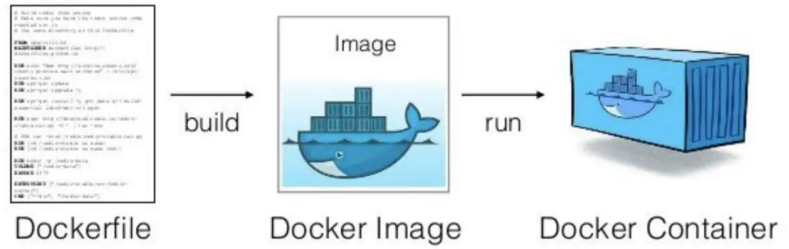
Prepare a docker file in the working directory. Save it as "Dockerfile"

Base image. Other wise use scratch

RUN => Commands to be executed while building the image

```
E: > CFDTEA_presentation > Demonstration_files > Demonstration_files >
1   FROM dolfinx/dolfinx:stable
2
3   RUN apt-get update && \
4       apt-get install -y nano && \
5       rm -rf /var/lib/apt/lists/*
6
7   RUN pip install --no-cache-dir matplotlib
8
9   CMD ["/bin/bash"]
10
```

CMD => Commands to be executed after making the container from of the image




https://www.mlwithramin.com/blog/docker

To build an image from the docker file

```
sudo docker build –t <imagename> .
```

List local images

```
sudo docker images
```

# Docker file of openFOAM

Base image
Other wise use scratch

RUN=Commands to executed while building the image

simple-openfoam-dockerfile / openfoam.org-release / **Dockerfile**

robinknowles Updated to accompany OnCFD Issue 009 - re-organised into dirs & updat...

Code | Blame | 25 lines (20 loc) · 694 Bytes

```
1   FROM ubuntu:focal
2
3   ENV DEBIAN_FRONTEND=noninteractive
4
5   RUN apt-get update \
6       && apt-get install -y \
7           vim \
8           ssh \
9           sudo \
10          wget \
11          software-properties-common ;\
12          rm -rf /var/lib/apt/lists/*
13
14  RUN useradd --user-group --create-home --shell /bin/bash foam ;\
15      echo "foam ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers
16
17  RUN sh -c "wget -O - http://dl.openfoam.org/gpg.key | apt-key add -" ;\
18      add-apt-repository http://dl.openfoam.org/ubuntu ;\
19      apt-get update ;\
20      apt-get install -y openfoam8 paraviewopenfoam56- ;\
21      rm -rf /var/lib/apt/lists/* ;\
22      echo "source /opt/openfoam8/etc/bashrc" >> ~foam/.bashrc ;\
23      echo "export OMPI_MCA_btl_vader_single_copy_mechanism=none" >> ~foam/.bashrc
24
25  USER foam
```

To build an image from the docker file

Sudo docker build –t imagefile

List local images

- sudo docker images

# More options in the Dockerfile

| Instruction | Description |
|---|---|
| ADD | Add local or remote files and directories. |
| ARG | Use build-time variables. |
| CMD | Specify default commands. |
| COPY | Copy files and directories. |
| ENTRYPOINT | Specify default executable. |
| ENV | Set environment variables. |
| EXPOSE | Describe which ports your application is listening on. |
| FROM | Create a new build stage from a base image. |
| HEALTHCHECK | Check a container's health on startup. |

| Instruction | Description |
|---|---|
| LABEL | Add metadata to an image. |
| MAINTAINER | Specify the author of an image. |
| ONBUILD | Specify instructions for when the image is used in a build. |
| RUN | Execute build commands. |
| SHELL | Set the default shell of an image. |
| STOPSIGNAL | Specify the system call signal for exiting a container. |
| USER | Set user and group ID. |
| VOLUME | Create volume mounts. |
| WORKDIR | Change working directory. |

Visit for more options in the docker file
https://docs.docker.com/reference/dockerfile/

# Working with docker container

THE UNIVERSITY OF QUEENSLAND AUSTRALIA

For more commands

- https://docs.docker.com/get-started/docker_cheatsheet.pdf
- https://www.geeksforgeeks.org/devops/docker-instruction-commands/
- https://docs.docker.com/reference/cli/docker/

List local images

`sudo docker images`

To create a container

`docker run --name <container_name> <image_name>`

To create a container by mounting a different drive or folder

`sudo docker run -it --mount src=</path_to_workingdirectory_on_machine>,target=/home/,type=bind <image_name>`

To list currently running containers:

`sudo docker ps`

List all docker containers (running and stopped)

`sudo docker ps –a`

Exiting the container

`exit`

Entering the container in terminal

`sudo docker exec -it dockername /bin/bash`

Start the container

`docker start <CONTAINER_ID>`

Stop the container

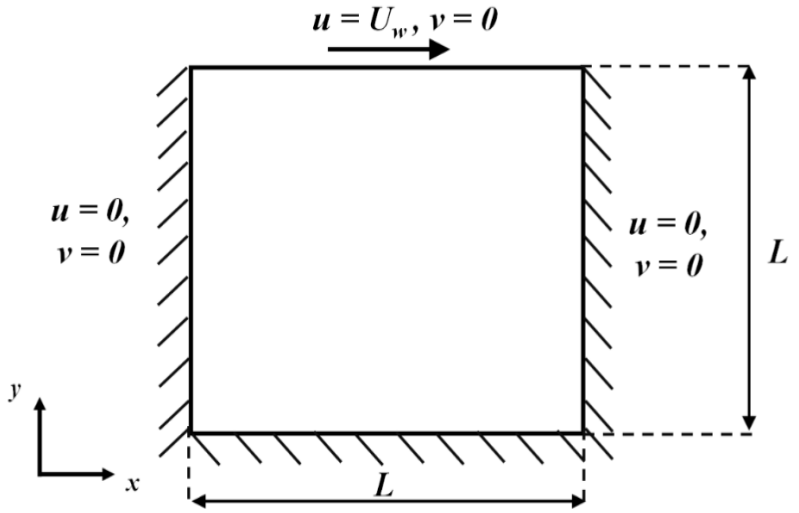
`docker stop <CONTAINER_ID>`

Removing a stopped container

`sudo docker rm < CONTAINER_ID >`

```
uqjabra1@aqua:~$ sudo docker images
REPOSITORY                          TAG       IMAGE ID       CREATED         SIZE
ghcr.io/fenics/dolfinx/dolfinx      nightly   d64be49a025f   13 months ago   3.23GB
ghcr.io/marchirschvogel/ambit       devenv    570744f6d8dd   14 months ago   3.36GB
ghcr.io/marchirschvogel/ambit       latest    f85217e0aa77   14 months ago   3.4GB
hello-world                         latest    74cc54e27dc4   16 months ago   10.1kB
ghcr.io/fenics/dolfinx/dolfinx      v0.9.0    cbc6e6edf697   19 months ago   2.79GB
uqjabra1@aqua:~$ ^C
uqjabra1@aqua:~$ ^C
uqjabra1@aqua:~$ ^C
uqjabra1@aqua:~$ ^C
uqjabra1@aqua:~$ sudo docker ps -a
CONTAINER ID   IMAGE                                      COMMAND       CREATED         STATUS                       PORTS     NAMES
8dae263f1019   ghcr.io/marchirschvogel/ambit              "/bin/bash"   13 months ago   Exited (137) 4 weeks ago             magical_mendeleev
5b448625718b   ghcr.io/marchirschvogel/ambit              "/bin/bash"   13 months ago   Exited (0) 13 months ago             nifty_herschel
0206f60ed51e   ghcr.io/fenics/dolfinx/dolfinx:v0.9.0      "/bin/bash"   13 months ago   Created                              pedantic_greider
6aff88a620af   ghcr.io/fenics/dolfinx/dolfinx:v0.9.0      "/bin/bash"   13 months ago   Up 13 months                         eloquent_jepsen
0c0231453a5b   ghcr.io/fenics/dolfinx/dolfinx:nightly     "/bin/bash"   13 months ago   Exited (1) 13 months ago             happy_ganguly
13ed96208c6a   ghcr.io/fenics/dolfinx/dolfinx:nightly     "/bin/bash"   13 months ago   Up 13 months                         busy_clarke
8d53e73915a7   ghcr.io/fenics/dolfinx/dolfinx:nightly     "/bin/bash"   13 months ago   Up 13 months                         nostalgic_nightingale
9ce83e3f420d   hello-world                                "/hello"      13 months ago   Exited (0) 13 months ago             sweet_mendel
uqjabra1@aqua:~$
```

# Example case: Lid driven cavity problem




$$\rho\left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u}\cdot\nabla\mathbf{u}\right) = -\nabla p + \nu\nabla^2\mathbf{u}$$

$$\nabla\cdot\mathbf{u} = 0$$

Apply the time discretisation: $\frac{\partial \mathbf{u}}{\partial t} \approx \frac{\mathbf{u}^{n+1}-\mathbf{u}^{n}}{\Delta t}$

$v =$ *velocity test function*
$q =$ *pressure test function*

Weak form: by multiplying with the test function, and integration by parts.

$$\int_\Omega \rho\left(\frac{\mathbf{u}^{n+1}-\mathbf{u}^{n}}{\Delta t}\right)\cdot\mathbf{v}\,d\Omega + \int_\Omega \rho\left((\mathbf{u}^{n}\cdot\nabla)\mathbf{u}^{n+1}\right)\cdot\mathbf{v}\,d\Omega + \int_\Omega \nu\,\nabla\mathbf{u}^{n+1}:\nabla\mathbf{v}\,d\Omega - \int_\Omega p^{n+1}\,(\nabla\cdot\mathbf{v})\,d\Omega + \int_\Omega(\nabla\cdot\mathbf{u}^{n+1})q\,d\Omega = 0.$$

Use Taylor Hood Elements to satisfy Ladyzhenskaya–Babuška–Brezzi (LBB) stability condition for incompressible fluids.
Velocity: quadratic (P2)
Pressure: linear (P1)

# Example case using FEniCS 2017-19 version

THE UNIVERSITY OF QUEENSLAND AUSTRALIA

```python
from dolfin import *
import matplotlib.pyplot as plt

# Define mesh and function space
mesh = UnitSquareMesh(50, 50)
P2 = VectorElement("P", mesh.ufl_cell(), 2)  # Velocity element
P1 = FiniteElement("P", mesh.ufl_cell(), 1)  # Pressure element
TH = MixedElement([P2, P1])
W = FunctionSpace(mesh, TH)                  #Mixed function space

# Define boundary conditions
inflow = Expression(("1.0", "0.0"), degree=2) # Inflow velocity

def lid(x, on_boundary):
    return on_boundary and near(x[1], 1.0)

def walls(x, on_boundary):
    return on_boundary and not near(x[1], 1.0)

bcu = [DirichletBC(W.sub(0), inflow, lid),
       DirichletBC(W.sub(0), Constant((0, 0)), walls)]
```

```python
# Time parameters
dt = 0.1
T = 2.0
t = 0.0 # Time step

# Functions
w = Function(W)
w_prev = Function(W)
u, p = split(w)
u_prev, p_prev = split(w_prev)
v, q = TestFunctions(W)

# Physical parameters
nu = 0.01
rho = 1.0

```

```python
# Variational formulation (weak form of Navier Stokes equation)
F = (rho * inner((u - u_prev)/dt, v) * dx
     + rho * inner(dot(u_prev, nabla_grad(u)), v) * dx
     + nu * inner(grad(u), grad(v)) * dx
     - p * div(v) * dx
     + div(u) * q * dx)

# Critical fix: Compute Jacobian and don't redefine problem
J = derivative(F, w, TrialFunction(W))
problem = NonlinearVariationalProblem(F, w, bcu, J=J)
solver = NonlinearVariationalSolver(problem)
print(solver.parameters)
# Add solver parameters
prm = solver.parameters
prm['newton_solver']['absolute_tolerance'] = 1e-12
prm['newton_solver']['relative_tolerance'] = 1e-14
prm['newton_solver']['maximum_iterations'] = 5
prm['newton_solver']['linear_solver'] = 'mumps'
```

```python
# Time-stepping
while t < T:
    w_prev.assign(w)
    solver.solve()
    t += dt
    #print(f"Time step t = {t:.3f}")

    # Save velocity and pressure to files for visualization in ParaView
    u, p = w.split()
    u_file << (u, t)
    p_file << (p, t)

```

*Source: https://github.com/Adari-Harsha/Lid_Driven_Cavity_Flow-*

# Example case using dolfinx version

THE UNIVERSITY OF QUEENSLAND AUSTRALIA

## FEniCS (dolfin) 2019 version

```python
from dolfin import *
import matplotlib.pyplot as plt

# Define mesh and function space
mesh = UnitSquareMesh(50, 50)
P2 = VectorElement("P", mesh.ufl_cell(), 2)  # Velocity element
P1 = FiniteElement("P", mesh.ufl_cell(), 1)  # Pressure element
TH = MixedElement([P2, P1])
W = FunctionSpace(mesh, TH)                  #Mixed function space

# Define boundary conditions
inflow = Expression(("1.0", "0.0"), degree=2) # Inflow velocity

def lid(x, on_boundary):
    return on_boundary and near(x[1], 1.0)

def walls(x, on_boundary):
    return on_boundary and not near(x[1], 1.0)

bcu = [DirichletBC(W.sub(0), inflow, lid),
       DirichletBC(W.sub(0), Constant((0, 0)), walls)]

# Time parameters
```

## FEniCSx(dolfinx) version

```python
from mpi4py import MPI
import numpy as np
import matplotlib.pyplot as plt

from dolfinx import mesh, fem, io
from dolfinx.fem import functionspace, Function, Constant, dirichletbc, locate_dofs_topological
from dolfinx.fem.petsc import NonlinearProblem
from dolfinx.io import XDMFFile
from basix.ufl import element, mixed_element
from ufl import (
    TestFunctions,
    split, inner, dot, grad, nabla_grad, div, dx,
)
from petsc4py import PETSc

# ------------------------------------------------------------------------
# Mesh
# ------------------------------------------------------------------------
msh = mesh.create_unit_square(MPI.COMM_WORLD, 50, 50)
```

```python
# ------------------------------------------------------------------------
# Mesh
# ------------------------------------------------------------------------
msh = mesh.create_unit_square(MPI.COMM_WORLD, 50, 50)

# ------------------------------------------------------------------------
# Mixed function space: Taylor-Hood P2/P1
# ------------------------------------------------------------------------
P2 = element("Lagrange", msh.basix_cell(), 2, shape=(msh.geometry.dim,))
P1 = element("Lagrange", msh.basix_cell(), 1)
TH = mixed_element([P2, P1])
W  = functionspace(msh, TH)

# ------------------------------------------------------------------------
# Boundary conditions
# ------------------------------------------------------------------------
# Locate facets on each boundary
tdim = msh.topology.dim
fdim = tdim - 1
msh.topology.create_connectivity(fdim, tdim)

# Lid: y == 1  (moving wall)
lid_facets   = mesh.locate_entities_boundary(msh, fdim, lambda x: np.isclose(x[1], 1.0))
# Walls: all other boundaries
wall_facets  = mesh.locate_entities_boundary(
    msh, fdim,
    lambda x: np.isclose(x[0], 0.0) | np.isclose(x[0], 1.0) | np.isclose(x[1], 0.0)
)
```

# Demonstration

# Part -B

# Singularity workflow

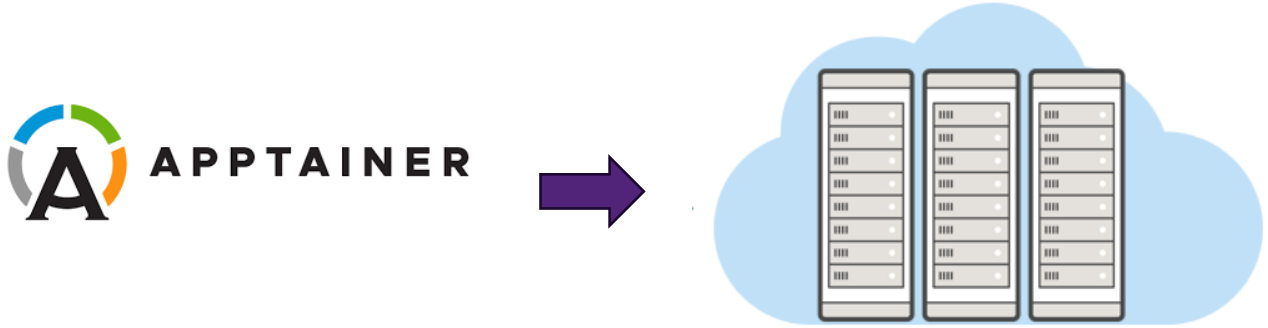
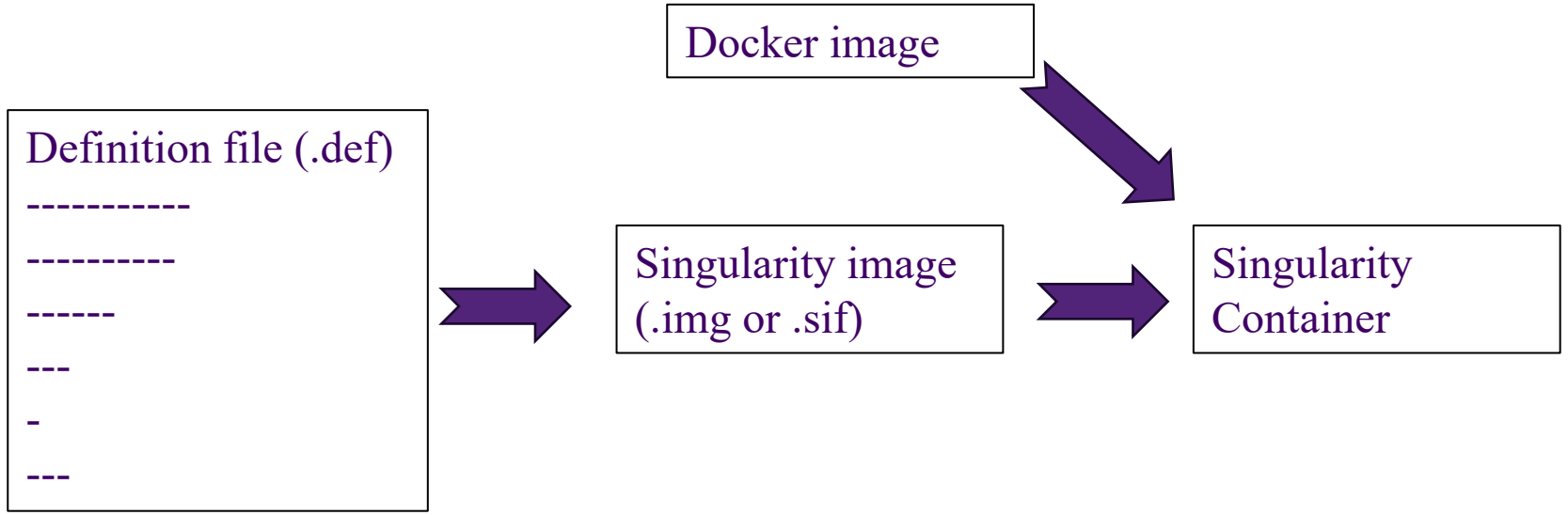



*Image source:shuttlestock*

# Installing singularity

To install the singularity (https://docs.sylabs.io/guides/3.5/user-guide/quick_start.html)

- Install dependencies
- Install Go
- Setup the environment
- Download singularity
- Compile the singularity using cmake




*Singularity images are easily portable*

# Structure of singularity definition file

Header

Sections

```
Bootstrap: library
From: ubuntu:22.04
Stage: build

%setup
    touch /file1
    touch ${SINGULARITY_ROOTFS}/file2

%files
    /file1
    /file1 /opt

%environment
    export LISTEN_PORT=54321
    export LC_ALL=C

%post
    apt-get update && apt-get install -y netcat
    NOW=`date`
    echo "export NOW=\"${NOW}\"" >> $SINGULARITY_ENVIRONMENT

%runscript
    echo "Container was created $NOW"
    echo "Arguments received: $*"
    exec echo "$@"

%startscript
    nc -lp $LISTEN_PORT

%test
    grep -q NAME=\"Ubuntu\" /etc/os-release
    if [ $? -eq 0 ]; then
        echo "Container base is Ubuntu as expected."
    else
        echo "Container base is not Ubuntu."
        exit 1
    fi

%labels
    Author myuser@example.com
    Version v0.0.1

%help
    This is a demo container used to illustrate a def file that uses all
    supported sections.
```

*https://docs.sylabs.io/guides/latest/user-guide/definition_files.html*

Two main parts

- Header
  - First specify the bootstrap agent to build the
  - Bootstrap options: library, docker, shub, oras, scratch….etc
  - Further entries in header are bootstrap specific
- Sections: All are optional
  - Setup
    - To be executed on host before making the OS.
  - Files
    - To copy files to the container

More examples: https://github.com/sylabs/examples

# Comparison of Dockerfile and Singularity definition file

| Function | Dockerfile Command | Singularity Definition File Command | Description |
|---|---|---|---|
| Base Image | FROM | BootStrap and From | Specifies the base image to use. |
| Labels/Metadata | LABEL | %labels | Adds metadata, such as author, version, etc. |
| Run Commands | RUN | %post | Executes commands during the build (e.g., installing software). |
| Environment Variables | ENV | %environment | Defines environment variables to be set in the container. |
| Default Command | CMD | %runscript | Sets the default command to run when the container is executed. |
| Expose Ports | EXPOSE | N/A | Docker-specific; Singularity containers don't expose ports. |
| Copy Files into Container | COPY or ADD | %files or cp inside %post | Copies files from the host into the container. |
| Working Directory | WORKDIR | cd command in %post or %runscript | Sets the working directory in the container. |
| Execute Command on Run | ENTRYPOINT | %runscript or exec command | Defines the main process that runs when the container starts. |
| Command for Tests | N/A | %test | Runs test commands to verify the build after it completes. |
| Custom Help Message | N/A | %help | Provides help documentation within the container. |
| Clean Up | RUN (e.g., apt-get clean) | Included within %post | Both involve cleaning up after installation to minimize size. |
| Mount Volumes | VOLUME | Bind mounts via -B flag or set in %post | Docker's VOLUME has no direct equivalent, but bind mounts are possible. |

*https://mivegec.pages.ird.fr/dainat/malbec-containers/pages/containers/containers-7-singularity/#from-a-singularity-definition-file*

# Examples of singularity definition file

Dockerfile

Base image

RUN

CMD

```
E: > CFDTEA_presentation > Demonstration_files > Demonstratio
1   FROM dolfinx/dolfinx:stable
2
3   RUN apt-get update && \
4       apt-get install -y nano && \
5       rm -rf /var/lib/apt/lists/*
6
7   RUN pip install --no-cache-dir matplotlib
8
9   CMD ["/bin/bash"]
10
```

Singularity definition file

Header

Sections

```
1   Bootstrap: docker
2   From: dolfinx/dolfinx:stable
3
4   %post
5       apt-get update && \
6       apt-get install -y nano && \
7       rm -rf /var/lib/apt/lists/*
8
9       pip install --no-cache-dir matplotlib
10
11  %runscript
12      exec /bin/bash "$@"
13
```

# Building a singularity container

Building an image from the file

sudo singularity build <container_name>.img buildfile

Running an image

Singularity run imagename

Runing, it loading a directory

Singularity shell --bind <path to load> imagename

**Singularity container from docker image**

docker save –o filename.tar imagename

Singularity build singularityimage name.img docker-archive://imagename

For eg: Singularity build fenics_stable.img docker-archive://fenics_stable:py3

# How to use singularity in Rangpur or Bunya

Keep the image  in your disk space.

Point to it in your job script.

```
#!/bin/bash

#SBATCH --nodes=1
#SBATCH --ntasks=12
#SBATCH --cpus-per-task=1
#SBATCH --ntasks-per-node=6
……
……

module load mpi/mpich-x86_64

mpirun singularity exec <path to the image> python filename.py
```

# Demonstration

# Drawbacks of container

| | General drawbacks | Current options/solutions |
|---|---|---|
| 1 | Security<br>Containers share the same host kernel, which can pose a security risk. If a container is compromised, it can potentially affect other containers on the same host. | container-specific security measures, such as container isolation and network segmentation, can mitigate this risk. |
| 2 | Complexity<br>Containers can be complex to set up and manage, especially in large-scale environments. | Container orchestration tools, such as Kubernetes, can help to simplify the process, but they can also add complexity. |
| 3 | Storage<br>Containers are designed to be stateless, meaning they don't store data or state. This can make managing persistent data, such as databases, difficult within a container environment. | Options such as persistent volumes, that can be used to manage data stored in a docker container environment. |
| 4 | Compatibility<br>Containers are designed to run on a specific container runtime, such as Docker or Kubernetes. This means that applications packaged in one container format may not be compatible with another container runtime. | There are tools available, such as container conversion tools, that can be used to convert containers between different formats. |

*Source:https://medium.com/@xcube_LABS/the-advantages-and-disadvantages-of-containers-57458db49aa2*

*Thank you….*

# Questions & discussion

For slides and demonstration files
https://github.com/jijoderick/CFD_Tea_discussion_UQ_SOMME_Jijo .git